

Post-Quantum Code Signing and Software Updates: Securing the Software Supply Chain with ML-DSA and SLH-DSA

Day 06 | Category: Software Supply Chain and Firmware Security

Date: 2026-07-13 | **DAILY**
MONITORING

As the quantum computing era approaches, the software supply chain has emerged as one of the most critical targets for cyber adversaries. At the heart of software security lies the digital signature, which ensures that operating system updates, application installations, and hardware firmware originate from trusted sources and have not been altered. While securing active internet traffic with hybrid key exchanges (such as X25519 + ML-KEM) is already widespread in browsers and SSH, transitioning the **software distribution and firmware signature** ecosystem to Post-Quantum Cryptography (PQC) presents unique challenges, particularly regarding signature longevity, verification speeds, and hardware storage constraints.

The Long-Term Integrity Problem: For typical web browsing, cryptographic keys are ephemeral and live for milliseconds. However, code signatures and firmware updates are designed to endure. If a medical device, an industrial PLC, or a vehicle's engine control unit is deployed in 2026 with a 15-year operational lifespan, its update verification keys must remain secure until 2041. A quantum computing breakthrough that compromises classical signing keys (RSA or ECDSA) retroactively invalidates the integrity of all past, present, and future updates, allowing attackers to inject malicious firmware without triggering bootloader warnings.

The Two Pillars of Post-Quantum Signatures: ML-DSA vs. SLH-DSA

To secure digital signatures against quantum threats, NIST has standardized two primary digital signature algorithms: **ML-DSA** (FIPS 204, based on module lattices) and **SLH-DSA** (FIPS 205, based on stateless hash structures). Each algorithm possesses distinct characteristics that dictate its applicability within the software supply chain:

- **ML-DSA (FIPS 204):** Based on the hardness of Module Learning With Errors (MLWE). It represents a general-purpose signature scheme. It features rapid signing, extremely fast verification speeds, and relatively compact public keys (~1.9 KB for ML-DSA-65) and signatures (~3.3 KB). It is the ideal candidate for high-frequency, large-scale general software

distribution (e.g., operating system package managers like APT, secure code-signing certificates, and app store updates).

- **SLH-DSA (FIPS 205):** Based strictly on stateless hash structures (SPHINCS+). It relies entirely on the collision resistance of underlying cryptographic hash functions (such as SHA-2 or SHAKE/SHA-3). Because SLH-DSA makes absolutely no structural algebraic assumptions, it is immune to future mathematical cryptanalysis that might theoretically compromise lattice-based schemes. However, it requires significantly longer signing times and produces large signatures (from ~7.8 KB to ~49 KB). Despite this overhead, its public key size is miniscule (~32 bytes), making it the gold standard for high-security applications, root certificate trust stores, and immutable hardware bootloaders.

Real-World Applicability in Secure Boot and OTA Updates

In practice, integrating post-quantum signatures into internet protocols and devices requires navigating strict physical and networking constraints:

1. Hardware Root of Trust and Secure Boot

During the device boot process, the secure bootloader verifies the initial firmware using a public key permanently burned into the chip's Read-Only Memory (ROM) or One-Time Programmable (OTP) fuses. This environment is highly resource-constrained. Storing a 2 KB ML-DSA public key in hardware registers is often impossible. Conversely, **SLH-DSA's 32-byte public key** fits easily into standard hardware fuses. Even though verifying an SLH-DSA signature requires more CPU cycles, the boot process happens infrequently, making the tiny public key size the deciding factor for hardware manufacturers.

2. Over-the-Air (OTA) Updates in IoT and Embedded Systems

IoT and embedded systems deployed across remote industrial networks (e.g., smart grids, telemetry sensors) often rely on low-bandwidth protocols like cellular, satellite, or LPWAN. Transporting a 17 KB SLH-DSA signature for a minor 50 KB configuration update represents a massive overhead that drains device battery and consumes network bandwidth. In these scenarios, developers are turning to **ML-DSA-44 or ML-DSA-65**, which offer a balanced trade-off between signature size and computational overhead, or deploying **hybrid signatures** that combine classical ECDSA with ML-DSA to ensure backward compatibility and quantum resistance.

3. Operating System Package Signing

Linux distributions and enterprise software vendors must sign millions of individual packages. Signing speed is critical for continuous integration and delivery (CI/CD) pipelines. While SLH-DSA can take hundreds of milliseconds to generate a single signature, ML-DSA can sign in less than a millisecond, making ML-DSA the only viable option for high-volume, automated software release cycles.

Comparing Signature Schemes for the Software Supply Chain

This table compares classical signature algorithms with the new post-quantum standards under FIPS 204 and FIPS 205, highlighting their technical trade-offs:

Algorithm Name	Type / Class	Public Key Size	Signature Size	Signing Speed	Verification Speed	Best Suited For
ECDSA (P-256)	Classical (Elliptic Curve)	~64 Bytes	~64 Bytes	Very Fast	Fast	No longer recommended (broken by Shor's Algorithm)
RSA-3072	Classical (Integer Factoring)	~384 Bytes	~384 Bytes	Slow	Very Fast	No longer recommended (broken by Shor's Algorithm)
ML-DSA-65 (FIPS 204)	Post-Quantum (Module Lattice)	1,952 Bytes	3,293 Bytes	Very Fast	Very Fast	General software signing, OS packages, app stores, CI/CD pipelines.
SLH-DSA-SHA2-128s (FIPS 205)	Post-Quantum (Stateless Hash)	32 Bytes	7,856 Bytes	Very Slow	Moderate	High-security firmware updates, ROM secure boot keys, Root CAs.
SLH-DSA-SHA2-128f (FIPS 205)	Post-Quantum (Stateless Hash)	32 Bytes	17,088 Bytes	Moderate	Fast	Applications requiring faster verification where signature size is secondary.

Daily Quiz: Test Your Understanding

1. **Why is SLH-DSA (FIPS 205) considered structurally safer than ML-DSA (FIPS 204) for long-term code signing root anchors, despite having larger signatures and slower signing speeds?**

A) Because SLH-DSA is stateful and requires tracking signature indexes, which prevents key replay attacks.

B) Because SLH-DSA is a stateless hash-based scheme relying only on secure hash functions, with zero structured algebraic (lattice) assumptions that could be compromised by future mathematical breakthroughs.

C) Because SLH-DSA is faster to sign on constrained microcontroller hardware.

Correct Answer: B. Since SLH-DSA depends solely on the collision resistance of standard hash functions (like SHA-2 or SHA-3), it does not rely on the complexity of lattice problems. It is mathematically simpler and less vulnerable to future algebraic cryptanalysis breakthroughs.

2. **Which characteristic of SLH-DSA makes it highly attractive for hardware secure boot ROM and OTP (One-Time Programmable) storage?**

A) Its massive signature size, which fills up the storage to prevent buffer overflows.

B) Its very fast signing speed, which accelerates factory programming lines.

C) Its exceptionally tiny public key size of just 32 bytes, which easily fits in limited chip fuse registers.

Correct Answer: C. High-security chips have extremely limited ROM/OTP space to store the trust anchor (public key). SLH-DSA's public key is only 32 bytes (equal to or smaller than classical ECC/RSA keys), making it incredibly easy to store in physical hardware fuses.

3. **A software developer is setting up an automated CI/CD pipeline that signs thousands of compiled microservices and packages per hour. Which PQC signature should they select?**

A) SLH-DSA-SHA2-128s, to minimize the public key size in their database.

B) ML-DSA-65, because its sub-millisecond signing and verification speeds prevent pipeline bottlenecks while maintaining quantum resistance.

C) Stateful XMSS (RFC 8391), because the continuous pipeline is perfect for tracking states across parallel server instances.

Correct Answer: B. Automated, high-frequency software build pipelines require fast signing speeds. ML-DSA provides rapid signing and verification with reasonable signature sizes, making it the practical choice for automated general-purpose software delivery.